```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from google.colab import files

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import (
    confusion_matrix,
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    classification_report
)

print("Libraries imported successfully!")
```

```
Libraries imported successfully!
```

```
uploaded = files.upload()
```

Choose Files WA_Fn-Us...er-Churn.csv

WA_Fn-UseC_-Telco-Customer-Churn.csv(text/csv) - 977501 bytes, last modified: 9/21/2026 - 100% done

```
Saving WA_Fn-UseC_-Telco-Customer-Churn.csv to WA_Fn-UseC_-Telco-Customer-Churn.csv
```

```
print("Column names:")
print(df.columns.tolist())
```

```
Column names:
['customerID', 'gender', 'SeniorCitizen', 'Partner', 'Dependents', 'tenure', 'PhoneService', 'MultipleLines'
```

```
df = pd.read_csv(filename)
```

```
print("Dataset Shape:", df.shape)

print("\nColumn Names:")
print(df.columns.tolist())

print("\nFirst 5 Rows:")
display(df.head())
```

```
Dataset Shape: (7043, 21)

Column Names:
['customerID', 'gender', 'SeniorCitizen', 'Partner', 'Dependents', 'tenure', 'PhoneService', 'MultipleLines'

First 5 Rows:
```

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	InternetServi
0	7590-VHVEG	Female	0	Yes	No	1	No	No phone service	D
1	5575-GNVDE	Male	0	No	No	34	Yes	No	D
2	3668-QPYBK	Male	0	No	No	2	Yes	No	D
3	7795-CFOCW	Male	0	No	No	45	No	No phone service	D
4	9237-HQITU	Female	0	No	No	2	Yes	No	Fiber o

5 rows × 21 columns

```
print("Dataset Information:")
df.info()
```

```
Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7043 entries, 0 to 7042
Data columns (total 21 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   customerID        7043 non-null   object
 1   gender            7043 non-null   object
 2   SeniorCitizen     7043 non-null   int64
 3   Partner           7043 non-null   object
 4   Dependents        7043 non-null   object
 5   tenure            7043 non-null   int64
 6   PhoneService      7043 non-null   object
 7   MultipleLines     7043 non-null   object
 8   InternetService   7043 non-null   object
 9   OnlineSecurity    7043 non-null   object
 10  OnlineBackup      7043 non-null   object
 11  DeviceProtection  7043 non-null   object
 12  TechSupport       7043 non-null   object
 13  StreamingTV       7043 non-null   object
 14  StreamingMovies   7043 non-null   object
 15  Contract          7043 non-null   object
 16  PaperlessBilling  7043 non-null   object
 17  PaymentMethod     7043 non-null   object
 18  MonthlyCharges    7043 non-null   float64
 19  TotalCharges      7043 non-null   object
 20  Churn             7043 non-null   object
dtypes: float64(1), int64(2), object(18)
memory usage: 1.1+ MB
```

```
print("Missing Values:")
print(df.isnull().sum())
```

```
Missing Values:
customerID          0
gender              0
SeniorCitizen       0
Partner             0
Dependents          0
tenure              0
PhoneService        0
```

```
MultipleLines       0
InternetService     0
OnlineSecurity      0
OnlineBackup        0
DeviceProtection    0
TechSupport         0
StreamingTV         0
StreamingMovies     0
Contract            0
PaperlessBilling    0
PaymentMethod       0
MonthlyCharges      0
TotalCharges        0
Churn               0
dtype: int64
```

```
for column in df.columns:

    if df[column].dtype == 'object':
        # Fill categorical columns with mode
        if df[column].isnull().sum() > 0:
            df[column] = df[column].fillna(df[column].mode()[0])

    else:
        # Fill numerical columns with median
        if df[column].isnull().sum() > 0:
            df[column] = df[column].fillna(df[column].median())

print("Missing values after preprocessing:")
print(df.isnull().sum())
```

```
Missing values after preprocessing:
customerID          0
gender              0
SeniorCitizen       0
Partner             0
Dependents          0
tenure              0
PhoneService        0
MultipleLines       0
InternetService     0
OnlineSecurity      0
OnlineBackup        0
DeviceProtection    0
TechSupport         0
StreamingTV         0
StreamingMovies     0
Contract            0
PaperlessBilling    0
PaymentMethod       0
MonthlyCharges      0
TotalCharges        0
Churn               0
dtype: int64
```

```
print("Duplicate rows before removal:", df.duplicated().sum())

df = df.drop_duplicates()

print("Duplicate rows after removal:", df.duplicated().sum())
print("Updated dataset shape:", df.shape)
```

```
Duplicate rows before removal: 0
Duplicate rows after removal: 0
Updated dataset shape: (7043, 21)
```

```
target_column = 'Churn'
```

```
print("Target column:", target_column)
print("\nTarget values:")
print(df[target_column].value_counts())
```

```
Target column: Churn

Target values:
Churn
No     5174
Yes    1869
Name: count, dtype: int64
```

```
df = df.drop(
    columns=['customerID'],
    errors='ignore'
)

print("Customer ID removed.")
print(df.columns.tolist())
```

```
Customer ID removed.
['gender', 'SeniorCitizen', 'Partner', 'Dependents', 'tenure', 'PhoneService', 'MultipleLines', 'InternetSer
```

```
X = df.drop(columns=[target_column])
y = df[target_column]

print("Number of features:", X.shape[1])

print("\nFeatures:")
print(X.columns.tolist())

print("\nTarget:")
print(target_column)
```

```
Number of features: 19

Features:
['gender', 'SeniorCitizen', 'Partner', 'Dependents', 'tenure', 'PhoneService', 'MultipleLines', 'InternetSer

Target:
Churn
```

```
X = pd.get_dummies(
    X,
    drop_first=True
)

print("Data after encoding:")
display(X.head())

print("Number of features after encoding:", X.shape[1])
```

```
Data after encoding:
```

	SeniorCitizen	tenure	MonthlyCharges	gender_Male	Partner_Yes	Dependents_Yes	PhoneService_Yes	Multipl phor
0	0	1	29.85	False	True	False	False	
1	0	34	56.95	True	False	False	True	
2	0	2	53.85	True	False	False	True	
3	0	45	42.30	True	False	False	False	
4	0	2	70.70	False	False	False	True	

5 rows × 6559 columns

```
Number of features after encoding: 6559
```

```
from sklearn.preprocessing import LabelEncoder

label_encoder = LabelEncoder()

y = label_encoder.fit_transform(y)

print("Target classes:")

for number, label in enumerate(label_encoder.classes_):
    print(number, "=", label)
```

```
Target classes:
0 = No
1 = Yes
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("Training data:", X_train.shape)
print("Testing data:", X_test.shape)
```

```
Training data: (5634, 6559)
Testing data: (1409, 6559)
```

```
from sklearn.tree import DecisionTreeClassifier

dt_model = DecisionTreeClassifier(
    criterion='gini',
    max_depth=5,
    random_state=42
)

dt_model.fit(X_train, y_train)

print("Decision Tree model trained successfully!")
```

```
Decision Tree model trained successfully!
```

```
y_pred = dt_model.predict(X_test)

print("Predictions generated successfully!")
```

```
print("First 20 predictions:")
print(y_pred[:20])
```

```
Predictions generated successfully!
First 20 predictions:
[0 1 0 0 0 0 0 0 0 0 0 0 1 0 0 0 1 0 0]
```

```
from sklearn.metrics import accuracy_score

accuracy = accuracy_score(
    y_test,
    y_pred
)

print("Accuracy:", round(accuracy, 4))
print("Accuracy:", round(accuracy * 100, 2), "%")
```

```
Accuracy: 0.7942
Accuracy: 79.42 %
```

```
from sklearn.metrics import precision_score, recall_score, f1_score

precision = precision_score(
    y_test,
    y_pred,
    zero_division=0
)

recall = recall_score(
    y_test,
    y_pred,
    zero_division=0
)

f1 = f1_score(
    y_test,
    y_pred,
    zero_division=0
)

print("Precision:", round(precision, 4))
print("Recall:", round(recall, 4))
print("F1-Score:", round(f1, 4))
```

```
Precision: 0.6312
Recall: 0.5401
F1-Score: 0.5821
```

```
from sklearn.metrics import classification_report

print(
    classification_report(
        y_test,
        y_pred,
        target_names=label_encoder.classes_,
        zero_division=0
    )
)
```

```
              precision    recall  f1-score   support

          No       0.84      0.89      0.86      1035
         Yes       0.63      0.54      0.58       374

    accuracy                           0.79      1409
   macro avg       0.74      0.71      0.72      1409
```

```
weighted avg       0.79      0.79      0.79      1409
```

```
from sklearn.metrics import confusion_matrix

cm = confusion_matrix(
    y_test,
    y_pred
)

print("Confusion Matrix:")
print(cm)
```

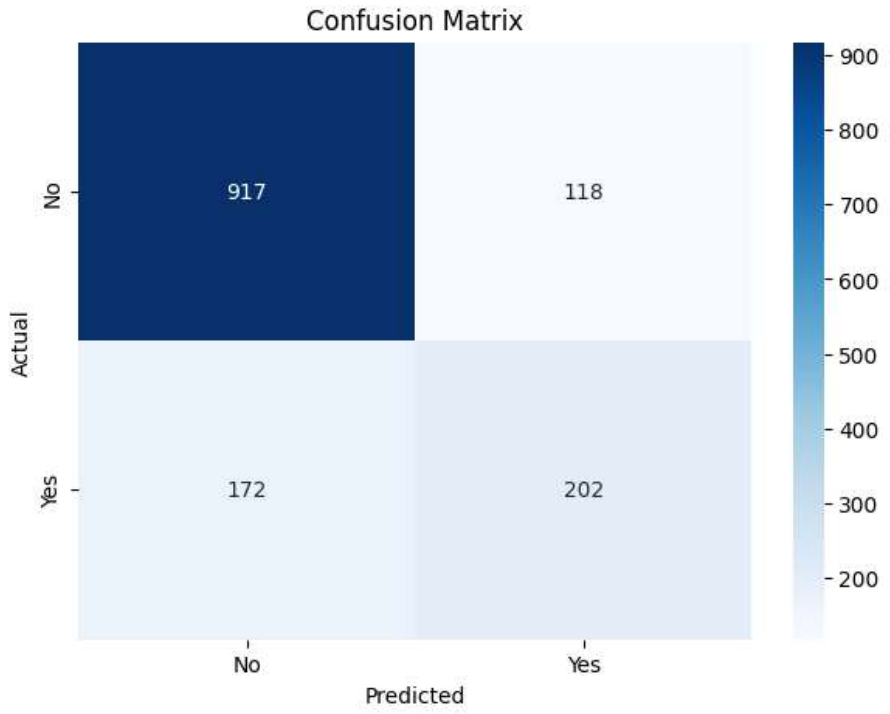
```
Confusion Matrix:
[[917 118]
 [172 202]]
```

```
plt.figure(figsize=(7, 5))

sns.heatmap(
    cm,
    annot=True,
    fmt='d',
    cmap='Blues',
    xticklabels=label_encoder.classes_,
    yticklabels=label_encoder.classes_
)

plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")

plt.show()
```




```
from sklearn.tree import plot_tree

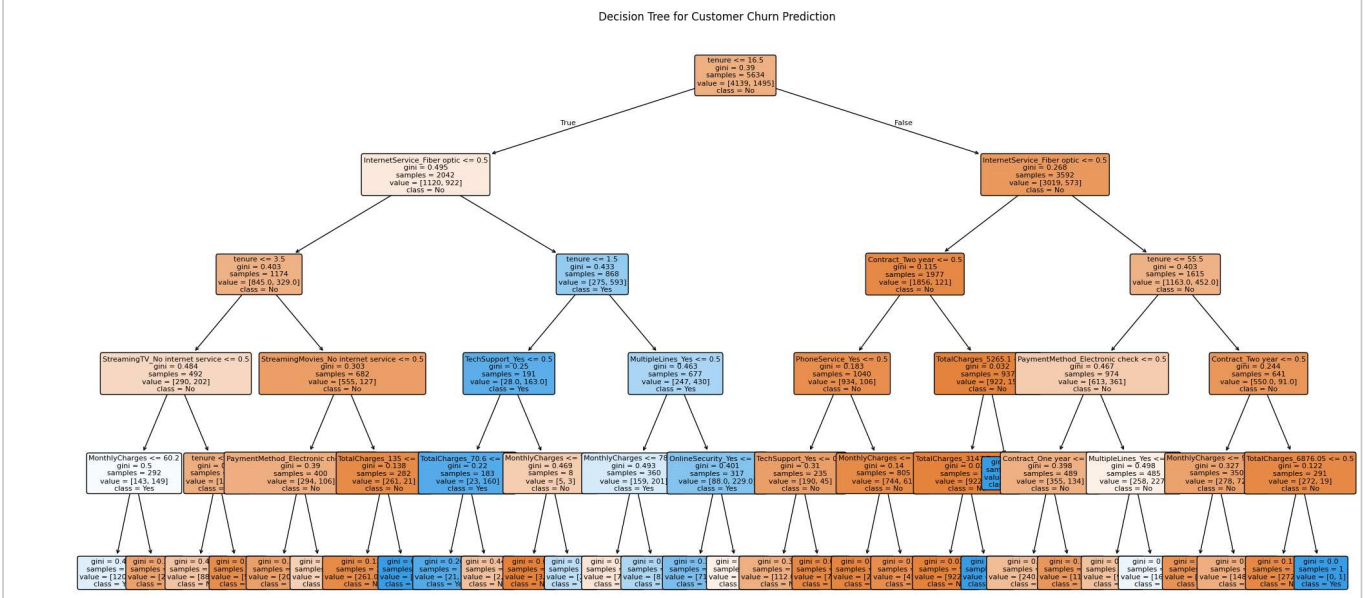
plt.figure(figsize=(25, 12))

plot_tree(
```

```
    dt_model,
    feature_names=X.columns,
    class_names=[
        str(x) for x in label_encoder.classes_
    ],
    filled=True,
    rounded=True,
    fontsize=8
)

plt.title("Decision Tree for Customer Churn Prediction")

plt.show()
```




```
feature_importance = pd.DataFrame({
    'Feature': X.columns,
    'Importance': dt_model.feature_importances_
})

feature_importance = feature_importance.sort_values(
    by='Importance',
    ascending=False
)

display(feature_importance)
```

	Feature	Importance
1	tenure	0.450207
9	InternetService_Fiber optic	0.357480
27	PaymentMethod_Electronic check	0.036336
2	MonthlyCharges	0.026674
8	MultipleLines_Yes	0.023264
...	...	...
2188	TotalCharges_250.05	0.000000
2187	TotalCharges_25.85	0.000000
2186	TotalCharges_25.8	0.000000
2185	TotalCharges_25.75	0.000000
2197	TotalCharges_2510.2	0.000000

6559 rows × 2 columns

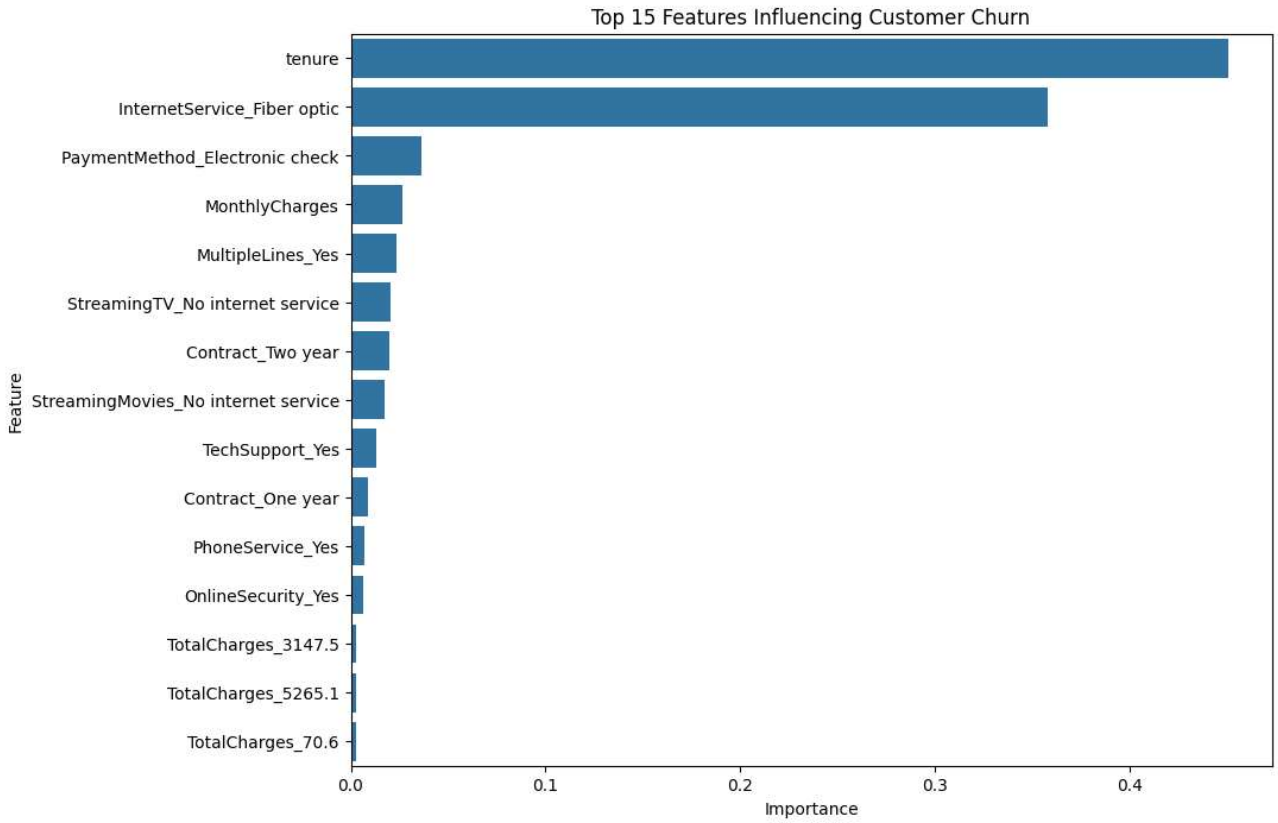
Next steps: Generate code with `feature_importance` New interactive sheet

```
plt.figure(figsize=(10, 8))

sns.barplot(
    data=feature_importance.head(15),
    x='Importance',
    y='Feature'
)

plt.title("Top 15 Features Influencing Customer Churn")
plt.xlabel("Importance")
plt.ylabel("Feature")

plt.show()
```




```
dt_entropy = DecisionTreeClassifier(
    criterion='entropy',
    max_depth=5,
    random_state=42
)

dt_entropy.fit(X_train, y_train)

y_pred_entropy = dt_entropy.predict(X_test)

entropy_accuracy = accuracy_score(
    y_test,
    y_pred_entropy
)

print(
    "Entropy Model Accuracy:",
    round(entropy_accuracy * 100, 2),
    "%"
)
```

```
Entropy Model Accuracy: 79.28 %
```

```
gini_accuracy = accuracy_score(
    y_test,
    y_pred
)

comparison = pd.DataFrame({
    'Criterion': [
        'Gini Index',
        'Entropy'
    ],
    'Accuracy': [
```

```
        gini_accuracy,
        entropy_accuracy
    ]
})

comparison['Accuracy (%)'] = (
    comparison['Accuracy'] * 100
)

display(comparison)
```

	Criterion	Accuracy	Accuracy (%)
0	Gini Index	0.794180	79.418027
1	Entropy	0.792761	79.276082

Next steps: Generate code with comparison | New interactive sheet

```
results = pd.DataFrame({
    'Metric': [
        'Accuracy',
        'Precision',
        'Recall',
        'F1-Score'
    ],
    'Score': [
        accuracy,
        precision,
        recall,
        f1
    ]
})

results['Percentage'] = results['Score'] * 100

display(results)
```

	Metric	Score	Percentage
0	Accuracy	0.794180	79.418027
1	Precision	0.631250	63.125000
2	Recall	0.540107	54.010695
3	F1-Score	0.582133	58.213256

Next steps: Generate code with results | New interactive sheet

```
Start coding or generate with AI.
```

```
Top 10 Factors Influencing Customer Churn
==================================================
1. tenure -> Importance: 0.4502
2. InternetService_Fiber optic -> Importance: 0.3575
3. PaymentMethod_Electronic check -> Importance: 0.0363
4. MonthlyCharges -> Importance: 0.0267
5. MultipleLines_Yes -> Importance: 0.0233
6. StreamingTV_No internet service -> Importance: 0.0207
7. Contract_Two year -> Importance: 0.0196
8. StreamingMovies_No internet service -> Importance: 0.0174
9. TechSupport_Yes -> Importance: 0.0130
10. Contract_One year -> Importance: 0.0086
```

Q1. What is classification, and how does it differ from regression?

Classification is a supervised machine learning technique used to predict categorical or discrete output classes.

For example, in customer churn prediction, the output can be: • Churn • No Churn

Regression is used to predict continuous numerical values.

Difference:

Classification: • Predicts categorical values. • Output consists of classes or categories. • Example: Churn prediction and spam detection.

Regression: • Predicts continuous numerical values. • Output is a numerical value. • Example: House price prediction and salary prediction.

Q2. Explain the working principle of the Decision Tree algorithm.

A Decision Tree is a supervised machine learning algorithm that makes predictions by dividing the dataset into smaller groups based on different feature values.

The main steps are:

1. The complete dataset is placed at the root node.
2. The algorithm selects the best feature for splitting the data.
3. The dataset is divided into smaller subsets.
4. The splitting process is repeated for the child nodes.
5. The process continues until a stopping condition is reached.
6. The final nodes are called leaf nodes and contain the prediction.

Decision Trees can use splitting criteria such as Gini Index or Entropy to select suitable splits.

Q3. Differentiate between Gini Index and Entropy as splitting criteria.

Gini Index and Entropy are two commonly used splitting criteria in Decision Tree classification.

Gini Index: • Measures the impurity of a node. • A lower Gini value indicates a purer node. • It is generally computationally simpler.

Entropy: • Measures the uncertainty or disorder in a node. • A lower entropy value indicates a purer node. • It uses logarithmic calculations.

Difference:

Gini Index:

- Based on impurity.
- Computationally simpler.
- Commonly used as a splitting criterion.

Entropy:

- Based on information or uncertainty.
- Uses logarithmic calculations.
- Uses information gain for selecting splits.

Both methods try to find splits that produce purer child nodes.

Q4. What is a Confusion Matrix? Explain its components.

A Confusion Matrix is a table used to evaluate the performance of a classification model.

It consists of four main components:

1. True Positive (TP): The model correctly predicts the positive class.
2. True Negative (TN): The model correctly predicts the negative class.
3. False Positive (FP): The model incorrectly predicts the positive class when the actual class is negative.
4. False Negative (FN): The model incorrectly predicts the negative class when the actual class is positive.

For customer churn prediction:

True Positive: A customer who actually churned is correctly predicted as churn.

True Negative: A customer who did not churn is correctly predicted as no churn.

False Positive: A customer who did not churn is incorrectly predicted as churn.

False Negative: A customer who actually churned is incorrectly predicted as no churn.

Q5. Define Accuracy, Precision, Recall, and F1-Score. Why are these metrics important?

Accuracy measures the proportion of correct predictions out of all predictions.

Accuracy = (TP + TN) / (TP + TN + FP + FN)

Precision measures how many of the observations predicted as positive are actually positive.

Precision = TP / (TP + FP)

Recall measures how many of the actual positive observations are correctly identified by the model.

Recall = TP / (TP + FN)

F1-Score is the harmonic mean of Precision and Recall.

F1-Score = 2 × (Precision × Recall) / (Precision + Recall)

These metrics are important because accuracy alone may not provide a complete picture of classification performance. Precision, Recall and F1-Score provide additional information about the model's predictions.

Q6. What is overfitting in a Decision Tree? How can it be reduced?

Overfitting occurs when a Decision Tree learns the training data too closely, including noise and small variations in the training dataset.

An overfitted tree may perform very well on training data but perform poorly on new or unseen data.

Overfitting can be reduced by:

1. Limiting the maximum depth of the tree.
2. Increasing the minimum number of samples required for a split.
3. Increasing the minimum number of samples required at a leaf.
4. Removing irrelevant features.
5. Applying pruning techniques.
6. Using appropriate train-test splitting and model evaluation.

Q7. Why is customer churn prediction important for businesses?

Customer churn prediction helps businesses identify customers who may discontinue their products or services.

Early identification of potential churn allows businesses to:

• Develop targeted customer retention strategies. • Improve customer satisfaction. • Provide personalized offers. • Identify factors related to customer churn. • Reduce potential revenue loss. • Improve customer retention.

Therefore, churn prediction helps organizations take appropriate actions before customers discontinue their services.